

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG



BÁO CÁO DỰ ÁN
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG XỬ LÝ TIẾNG NÓI ĐA NĂNG KẾT
HỢP KHỦ NHIEU VÀ BIẾN ĐỔI GIỌNG NÓI

Sinh viên thực hiện: Nguyễn Đức Giang

Môn học: Xử lý tín hiệu số

Giảng viên môn học: TS. Trần Thị Thúy Quỳnh

Mã nguồn: [gnett16/ProjectXLTHS](#)

HÀ NỘI – 2026

LỜI MỞ ĐẦU

Trong bối cảnh kỷ nguyên số và sự phát triển mạnh mẽ của trí tuệ nhân tạo, xử lý tín hiệu tiếng nói đã trở thành một lĩnh vực quan trọng với nhiều ứng dụng thực tiễn như viễn thông, nhận dạng giọng nói, trợ lý ảo, hệ thống giải trí và công nghệ điện ảnh [1]. Không chỉ dừng lại ở việc giảm nhiễu hay cải thiện chất lượng âm thanh, các kỹ thuật xử lý tiếng nói hiện đại còn cho phép thay đổi đặc trưng sinh học và âm sắc của giọng nói, mở ra nhiều hướng ứng dụng mới trong công nghệ âm thanh số.

Đồ án này được thực hiện nhằm hệ thống hóa và vận dụng các kiến thức nền tảng của môn Xử lý tín hiệu số (Digital Signal Processing – DSP) vào việc xây dựng một hệ thống xử lý tiếng nói hoàn chỉnh trên nền tảng MATLAB. Hệ thống được thiết kế theo một pipeline xử lý liên tục, bắt đầu từ quá trình thu nhận tín hiệu âm thanh thông qua microphone. Sau khi thu âm, tín hiệu được đưa qua giai đoạn tiền xử lý để loại bỏ các thành phần không mong muốn như độ lệch DC (DC offset) và nhiễu xung ngắn (Pop noise), giúp cải thiện chất lượng tín hiệu đầu vào.

Tiếp theo, hệ thống áp dụng bộ lọc FIR thông dải kết hợp với thuật toán trừ phổ (Spectral Subtraction) nhằm tách và làm nổi bật tín hiệu tiếng nói trong môi trường có nhiễu. Sự kết hợp giữa các phương pháp lọc trong miền thời gian và miền tần số giúp nâng cao hiệu quả khử nhiễu, đồng thời duy trì tính tự nhiên của giọng nói.

Điểm nổi bật của đồ án nằm ở khối biến đổi giọng nói. Thông qua việc tác động lên phổ tần số và cấu trúc thời gian của tín hiệu, hệ thống có khả năng mô phỏng sự thay đổi chiều dài ống thanh âm sinh học, từ đó thực hiện chuyển đổi giọng nam sang giọng nữ một cách tự nhiên. Ngoài ra, hệ thống còn có thể tạo hiệu ứng giọng robot bằng kỹ thuật điều chế tín hiệu kết hợp lượng tử hóa, mang lại âm sắc điện tử đặc trưng [2].

Để đánh giá hiệu quả của các thuật toán DSP được áp dụng, tín hiệu đầu ra được phân tích và trực quan hóa bằng nhiều công cụ khác nhau như biểu diễn miền thời gian, phân tích phổ tần số bằng FFT và biểu diễn phổ thời gian – tần số bằng Spectrogram (STFT). Các kết quả này cho thấy khả năng ứng dụng hiệu quả của các phương pháp xử lý tín hiệu số trong việc giải quyết các bài toán thực tế liên quan đến xử lý âm thanh và biến đổi giọng nói.

LỜI MỞ ĐẦU.....	1
CHƯƠNG 1: TỔNG QUAN VỀ XỬ LÝ TÍN HIỆU TIẾNG NÓI	4
1.1. Mô hình tạo tiếng nói và đặc trưng tần số	4
1.2. Định lý lấy mẫu và hiện tượng Aliasing.....	4
1.3. Mục tiêu và cấu trúc của đồ án.....	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. Tín hiệu âm thanh số và quá trình số hóa	6
2.1.1. Lấy mẫu và định lý Nyquist - Shannon.....	6
2.1.2. Lượng tử hóa	6
2.2. Biến đổi Fourier rời rạc (DFT) và FFT	6
2.3. Biến đổi Fourier thời gian ngắn (STFT) và Spectrogram.....	7
2.4. Thiết kế bộ lọc FIR	8
2.5. Kỹ thuật biến đổi cao độ.....	8
2.6. Lý thuyết Formant và biến đổi giọng nữ	9
2.7. Kỹ thuật Ring Modulation tạo giọng robot.....	9
CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG.....	10
3.1. Tổng Quan Kiến Trúc	10
3.2. Danh Sách và Chức Năng Các Mô-đun	11
3.3. Cơ Chế Xử Lý Lỗi và Tự Phục Hồi.....	12
3.4. Tham Số Hệ Thống	13
CHƯƠNG 4: CÀI ĐẶT CHI TIẾT CÁC MÔ-ĐUN	13
4.1. Mô-đun Thu Âm và Tiền Xử Lý (VoiceProcessor_Main.m)	13
4.1.1. Thu Âm	13
4.1.2. Khử Lỗi Phản Cứng Microphone.....	14
4.1.3. Kiểm Tra Tín Hiệu.....	14
4.2. Mô-đun Lọc Nhiễu – vp_spectralSubtraction.m.....	14
4.2.1. Thiết Kế Bộ Lọc	14
4.2.2. Áp Dụng Bộ Lọc và Chuẩn Hóa	14
4.3. Mô-đun Biến Đổi Cao Độ – vp_pitchShift.m.....	15
4.3.1. Cách 1: Audio Toolbox (shiftPitch).....	15

4.3.2. Cách 2: Fallback bằng Resample	15
4.4. Mô-đun Giọng Nữ – vp_femaleVoice.m.....	15
4.5. Mô-đun Giọng Robot – vp_robotVoice.m	16
4.6. Mô-đun Trực Quan Hóa – vp_plotAll.m	16
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	16
5.1. Môi Trường Kiểm Thử.....	16
5.2. Kết Quả Thu Âm và Tiền Xử Lý	17
5.3. Kết Quả Lọc Nhiễu	17
5.4. Kết Quả Biến Đổi Giọng Nói.....	18
5.4.1. Giọng Nam.....	18
5.4.2. Giọng Nữ.....	18
5.4.3. Giọng Robot.....	18
5.5. Phân Tích Spectrogram So Sánh.....	18
5.6. File Output.....	18
5.7. Đánh Giá Tổng Thể	19
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
6.1. Kết luận.....	20
6.2. Hướng phát triển.....	20
6.2.1. Nâng cao chất lượng xử lý	20
6.2.2. Mở rộng tính năng hệ thống	21
6.2.3. Ứng Dụng Thực Tiễn	21
TÀI LIỆU THAM KHẢO	22
PHỤ LỤC: MÃ NGUỒN MATLAB	23

CHƯƠNG 1: TỔNG QUAN VỀ XỬ LÝ TÍN HIỆU TIẾNG NÓI

1.1. Mô hình tạo tiếng nói và đặc trưng tần số

Theo lý thuyết âm học tiếng nói do G. Fant đề xuất, quá trình tạo tiếng nói của con người có thể được mô hình hóa bằng mô hình Nguồn - Lọc. Trong mô hình này, nguồn kích thích là luồng không khí từ phổi đi qua thanh quản, tạo ra dao động tuần hoàn đối với âm hữu thanh hoặc tín hiệu nhiễu đối với âm vô thanh. Tín hiệu sau đó được định hình bởi hệ thống cộng hưởng của khoang miệng, khoang mũi và vòm họng, đóng vai trò như một bộ lọc.

Tín hiệu tiếng nói mang nhiều đặc trưng quan trọng trong miền tần số, trong đó nổi bật nhất là tần số cơ bản và các đỉnh cộng hưởng Formant.

- **Tần số cơ bản (F_0):**
Đây là tần số dao động của dây thanh âm, quyết định độ cao thấp (Pitch) của giọng nói. Ở nam giới trưởng thành, giá trị F_0 thường nằm trong khoảng từ 85 Hz đến 180 Hz do dây thanh âm dài và dày hơn. Trong khi đó, nữ giới có F_0 cao hơn, dao động trong khoảng từ 165 Hz đến 255 Hz [1].
- **Các đỉnh cộng hưởng (Formants - $F_1, F_2, F_3, \dots$):**
Formant là các đỉnh năng lượng xuất hiện trên phổ tín hiệu tiếng nói do hiện tượng cộng hưởng trong ống thanh âm. Các Formant quyết định âm sắc (Timbre) và đóng vai trò quan trọng trong việc phân biệt nguyên âm. Do chiều dài ống thanh âm của nữ giới ngắn hơn nam giới nên các Formant của giọng nữ thường cao hơn khoảng 15 - 20% [4].

Năng lượng của tín hiệu tiếng nói chủ yếu tập trung trong dải tần từ 300 Hz đến 3400 Hz. Đây cũng là dải tần được tiêu chuẩn hóa trong các hệ thống viễn thông nhằm đảm bảo khả năng truyền tải tiếng nói hiệu quả với băng thông tối ưu. Vì vậy, trong đồ án này, dải tần 300 - 3400 Hz được lựa chọn làm cơ sở cho việc thiết kế các bộ lọc xử lý tín hiệu.

1.2. Định lý lấy mẫu và hiện tượng Aliasing

Để máy tính có thể xử lý tín hiệu âm thanh liên tục trong thực tế, tín hiệu cần được chuyển đổi sang dạng số thông qua hai quá trình chính là lấy mẫu (Sampling) và lượng tử hóa (Quantization) [2].

Theo định lý lấy mẫu Nyquist - Shannon, một tín hiệu tương tự có tần số cực đại là $f_{\max}$ có thể được khôi phục hoàn toàn nếu tần số lấy mẫu f_s thỏa mãn điều kiện:

$$f_s > 2 \cdot f_{\max}$$

Nếu điều kiện này không được đáp ứng, hiện tượng chồng phổ (Aliasing) sẽ xảy ra, làm cho các thành phần tần số cao bị gập xuống dải tần thấp, gây méo tín hiệu và mất thông tin.

Tai người có khả năng cảm nhận âm thanh trong khoảng từ 20 Hz đến 20 kHz. Do đó, để đảm bảo khả năng tái tạo âm thanh đầy đủ và hạn chế hiện tượng Aliasing, hệ thống trong đồ án sử dụng tần số lấy mẫu:

$$f_s = 44100 \text{ Hz}$$

Đây là chuẩn lấy mẫu phổ biến trong hệ thống CD Audio và nhiều ứng dụng xử lý âm thanh số hiện nay.

1.3. Mục tiêu và cấu trúc của đồ án

Đồ án hướng đến việc xây dựng một hệ thống xử lý tiếng nói hoàn chỉnh trên nền tảng MATLAB với các mục tiêu chính như sau:

- Xây dựng hệ thống thu nhận và xử lý tín hiệu âm thanh trực tiếp từ microphone.
- Thiết kế bộ lọc số FIR thông dải nhằm loại bỏ nhiễu ngoài băng và khử thành phần DC Offset phát sinh từ phần cứng.
- Cài đặt thuật toán biến đổi Pitch kết hợp dịch chuyển Formant nhằm chuyển đổi tự nhiên giữa giọng nam và giọng nữ.
- Ứng dụng kỹ thuật Ring Modulation kết hợp Comb Filter để tạo hiệu ứng giọng robot.
- Trực quan hóa tín hiệu bằng các công cụ phân tích DSP

Thông qua các nội dung trên, đồ án không chỉ giúp củng cố kiến thức lý thuyết về xử lý tín hiệu số mà còn minh họa khả năng ứng dụng DSP vào các hệ thống xử lý âm thanh thực tế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tín hiệu âm thanh số và quá trình số hóa

Âm thanh tự nhiên là tín hiệu liên tục, biểu diễn dưới dạng dao động áp suất không khí theo thời gian. Để máy tính có thể lưu trữ và xử lý tín hiệu âm thanh, tín hiệu cần được chuyển đổi sang dạng số thông qua hai quá trình chính là lấy mẫu và lượng tử hóa.

2.1.1. Lấy mẫu và định lý Nyquist - Shannon

Theo định lý lấy mẫu Nyquist - Shannon, để có thể tái tạo chính xác một tín hiệu tương tự có tần số cực đại là f_{max} , tần số lấy mẫu f_s phải thỏa mãn điều kiện:

$$f_s \geq 2 * f_{max}$$

Nếu điều kiện này không được đáp ứng, hiện tượng chồng phổ (Aliasing) sẽ xảy ra, làm cho các thành phần tần số cao bị gập xuống dải tần thấp, gây méo tín hiệu và mất thông tin.

Giọng nói con người có dải tần hữu ích chủ yếu từ khoảng 80 Hz đến 8000 Hz. Trong đó, tần số lấy mẫu $f_s = 44100$ Hz được sử dụng nhằm đảm bảo chất lượng âm thanh cao và loại bỏ nguy cơ Aliasing. Đây cũng là chuẩn lấy mẫu phổ biến trong hệ thống CD Audio và các ứng dụng xử lý âm thanh số hiện đại.

2.1.2. Lượng tử hóa

Sau quá trình lấy mẫu, biên độ tín hiệu được chuyển đổi thành các giá trị rời rạc thông qua quá trình lượng tử hóa. Hệ thống sử dụng độ phân giải 16-bit, tương ứng với 65536 mức lượng tử.

Với độ phân giải này, tỉ số tín hiệu trên nhiễu lượng tử hóa đạt xấp xỉ 98 dB, đủ để biểu diễn chính xác các đặc trưng của tín hiệu tiếng nói và đảm bảo chất lượng âm thanh trong quá trình xử lý.

2.2. Biến đổi Fourier rời rạc (DFT) và FFT

Biến đổi Fourier rời rạc (Discrete Fourier Transform - DFT) là công cụ cơ bản trong phân tích phổ tín hiệu số. Với tín hiệu rời rạc $x[n]$ gồm N mẫu, DFT được định nghĩa như sau:

$$X[k] = \sum x[n] * \exp(-j2\pi kn/N)$$

với $k = 0, 1, \dots, N-1$.

DFT cho phép chuyển tín hiệu từ miền thời gian sang miền tần số, giúp xác định các thành phần tần số có trong tín hiệu.

Tuy nhiên, việc tính trực tiếp DFT có độ phức tạp tính toán lớn $O(N^2)$. Vì vậy, thuật toán FFT được sử dụng để giảm độ phức tạp xuống còn $O(N \log N)$, giúp việc phân tích phổ trong thời gian thực trở nên khả thi.

2.3. Biến đổi Fourier thời gian ngắn (STFT) và Spectrogram

Tín hiệu tiếng nói là tín hiệu không dừng (Non-Stationary Signal), nghĩa là các đặc trưng phổ thay đổi theo thời gian. Do đó, việc sử dụng FFT trên toàn bộ tín hiệu không phản ánh được sự biến đổi này.

Để giải quyết vấn đề trên, hệ thống sử dụng biến đổi Fourier thời gian ngắn (Short-Time Fourier Transform - STFT). Phương pháp này chia tín hiệu thành các khung ngắn, áp dụng hàm cửa sổ cho từng khung rồi thực hiện FFT trên từng đoạn tín hiệu.

Biểu thức của STFT được viết như sau:

$$X(m,k) = \sum x[n] * w[n - mH] * \exp(-j2\pi kn/N)$$

Trong đó:

- $w[n]$ là hàm cửa sổ.
- m là chỉ số khung.
- H là bước nhảy giữa các khung (Hop Size).
- N là số điểm FFT.

Trong đồ án, cửa sổ Hann được sử dụng nhằm giảm hiện tượng rò rỉ phổ. Các thông số được lựa chọn gồm:

- Kích thước cửa sổ: 512 mẫu.
- Overlap: 256 mẫu.

- NFFT: 1024 điểm.

Các thông số này giúp cân bằng giữa độ phân giải thời gian và độ phân giải tần số trong Spectrogram.

2.4. Thiết kế bộ lọc FIR

Bộ lọc FIR là bộ lọc số có đáp ứng xung hữu hạn, được mô tả bởi phương trình sai phân:

$$y[n] = \sum b[k] * x[n-k]$$

Bộ lọc FIR có ưu điểm lớn là có thể thiết kế với đặc tính pha tuyến tính, giúp tránh méo pha và bảo toàn hình dạng sóng của tín hiệu tiếng nói trong miền thời gian.

Trong đồ án, các bộ lọc FIR được sử dụng gồm:

- Bộ lọc thông dải bậc 100 với dải thông từ 300 Hz đến 3400 Hz trong mô-đun vp_spectralSubtraction nhằm loại bỏ nhiễu ngoài dải tiếng nói.
- Bộ lọc thông cao bậc 60 với tần số cắt 3500 Hz trong mô-đun vp_femaleVoice nhằm tạo hiệu ứng hơi thở (Breathiness).
- Bộ lọc thông dải bậc 60 trong dải từ 1500 Hz đến 3500 Hz nhằm tăng cường vùng tần số sáng đặc trưng của giọng nữ.

2.5. Kỹ thuật biến đổi cao độ

Cao độ của giọng nói liên quan trực tiếp đến tần số cơ bản F0 của tín hiệu.

Trong hệ âm nhạc phương Tây, mỗi bán cung tương ứng với tỉ lệ tần số:

$$f_{\text{new}} = f_{\text{original}} * 2^{(n/12)}$$

Trong đó:

- n là số bán cung cần thay đổi.
- f_original là tần số ban đầu.
- f_new là tần số sau khi biến đổi.

Trong đồ án, phương pháp biến đổi cao độ được thực hiện bằng kỹ thuật tái lấy mẫu. Khi thay đổi tốc độ phát lại tín hiệu:

- Nếu tín hiệu được phát nhanh hơn, Pitch tăng lên.
- Nếu tín hiệu được phát chậm hơn, Pitch giảm xuống.

Hàm `resample()` của MATLAB được sử dụng với các tỉ lệ hữu tỉ p/q nhằm đảm bảo độ chính xác trong quá trình nội suy và lấy mẫu lại tín hiệu.

Phương pháp này có ưu điểm là đơn giản, dễ triển khai và phù hợp với các hệ thống xử lý tiếng nói thời gian thực.

2.6. Lý thuyết Formant và biến đổi giọng nữ

Formant là các vùng cộng hưởng của ống thanh âm, xuất hiện dưới dạng các đỉnh năng lượng trong phổ tín hiệu tiếng nói.

Do ống thanh âm của nữ giới ngắn hơn nam giới nên các Formant của giọng nữ thường nằm ở tần số cao hơn khoảng 15%.

Giá trị Formant điển hình của giọng nam và giọng nữ được trình bày như sau:

Formant	Giọng nữ	Giọng nam
F1	500 – 800 hz	700 – 1100 hz
F2	1000 – 1500 hz	1300 – 2000 hz
F3	2000 – 2800 hz	2500 – 3500 hz

Trong đồ án, hiệu ứng biến đổi Formant được thực hiện bằng cách co ngắn tín hiệu khoảng 15% thông qua kỹ thuật Resampling với tỉ lệ xấp xỉ 1/1.15. Điều này làm dịch chuyển các Formant lên vùng tần số cao hơn, mô phỏng đặc tính của giọng nữ.

2.7. Kỹ thuật Ring Modulation tạo giọng robot

Ring Modulation là kỹ thuật điều chế bằng cách nhân tín hiệu tiếng nói với một sóng mang hình sin.

Biểu thức toán học được viết như sau:

$$y(t) = x(t) * \sin(2\pi f_{\text{carrier}} * t)$$

Theo định lý điều chế, phép nhân trong miền thời gian tương đương convolution trong miền tần số. Kết quả là mỗi thành phần tần số f trong $x(t)$ sẽ tạo ra hai thành phần mới tại $(f + f_{\text{carrier}})$ và $|f - f_{\text{carrier}}|$. Hiệu ứng này tạo ra âm thanh kim loại, máy móc đặc trưng. Với $f_{\text{carrier}} = 400\text{Hz}$, hiệu ứng robot đặc trưng nhất cho dải giọng nói 300-3000Hz.

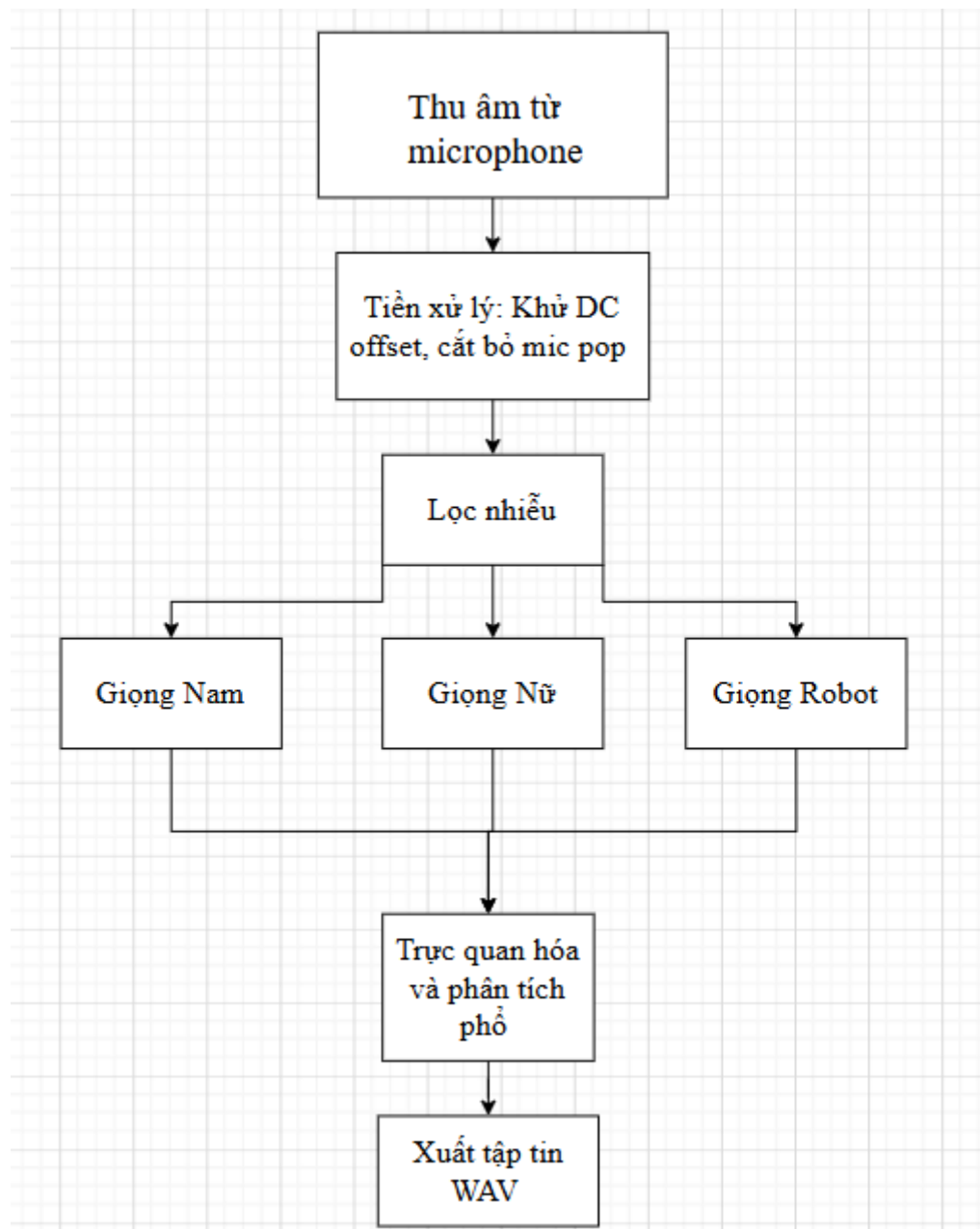
CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG

3.1. Tổng Quan Kiến Trúc

Hệ thống được thiết kế theo mô hình pipeline – chuỗi xử lý tuyến tính. Tín hiệu âm thanh đi qua từng tầng xử lý độc lập, đầu ra của tầng trước là đầu vào của tầng sau. Kiến trúc này có ưu điểm: dễ debug, dễ mở rộng, và mỗi mô-đun có thể kiểm thử độc lập.

Luồng xử lý chính:

1. Thu âm từ microphone (audiorecorder, 44100Hz, 16-bit, Mono)
2. Tiền xử lý: Khử DC offset, cắt bỏ mic pop (0.1 giây đầu)
3. Lọc nhiễu: Bandpass FIR (300-3400Hz) – `vp_spectralSubtraction`
4. Biến đổi giọng nói song song: Giọng Nam, Giọng Nữ, Giọng Robot
5. Trực quan hóa và phân tích phổ – `vp_plotAll`
6. Xuất tập tin WAV – `audio_output`



Hình 1: Pipeline hệ thống

3.2. Danh Sách và Chức Năng Các Mô-đun

Tên File	Chức Năng
VoiceProcessor_Main.m	Script chính điều phối toàn bộ pipeline, thu âm và lưu file
vp_spectralSubtraction.m	Lọc nhiễu bằng bộ lọc FIR thông dải, chuẩn hóa biên độ

vp_pitchShift.m	Biến đổi cao độ theo đơn vị bán cung (Audio Toolbox hoặc fallback)
vp_femaleVoice.m	Biến đổi giọng nữ
vp_robotVoice.m	Tạo hiệu ứng robot bằng ring modulation với sóng mang 400Hz
vp_plotAll.m	Vẽ đồ thị 3 cấp: dạng sóng, phổ FFT, Spectrogram STFT và lưu ảnh

3.3. Cơ Chế Xử Lý Lỗi và Tự Phục Hồi

Hệ thống được thiết kế với nhiều cơ chế bảo vệ và tự phục hồi nhằm đảm bảo tính ổn định và độ tin cậy trong quá trình xử lý tín hiệu âm thanh.

Kiểm tra tín hiệu đầu vào:

Trước khi xử lý, hệ thống kiểm tra biên độ cực đại của tín hiệu đầu vào. Nếu giá trị $\max(|x|)$ nhỏ hơn ngưỡng $1e-6$, tín hiệu được xem là không hợp lệ hoặc gần như không chứa dữ liệu âm thanh. Khi đó, hệ thống sẽ hiển thị thông báo lỗi và dừng quá trình xử lý nhằm tránh phát sinh lỗi ở các bước tiếp theo.

Cơ chế dự phòng trong mô-đun vp_pitchShift:

Trong quá trình biến đổi cao độ, hệ thống ưu tiên sử dụng hàm shiftPitch thuộc Audio Toolbox của MATLAB để đạt chất lượng xử lý tốt hơn. Tuy nhiên, nếu Audio Toolbox không được cài đặt hoặc hàm shiftPitch không khả dụng, hệ thống sẽ tự động chuyển sang phương pháp Resampling để đảm bảo chương trình vẫn hoạt động bình thường.

Kiểm tra tín hiệu sau khử nhiễu:

Sau quá trình lọc và trừ phổ, hệ thống kiểm tra năng lượng của tín hiệu đầu ra x_{clean} . Nếu tín hiệu sau xử lý có biên độ quá nhỏ hoặc gần bằng 0, hệ thống sẽ phát cảnh báo do khả năng bộ lọc đã loại bỏ quá nhiều thông tin hữu ích. Trong trường hợp này, tín hiệu gốc sẽ được sử dụng thay thế nhằm đảm bảo chất lượng âm thanh đầu ra.

Cơ chế chống clipping khi lưu file:

Trước khi ghi tín hiệu ra file âm thanh, hàm safeWrite() thực hiện chuẩn hóa biên độ tín hiệu về khoảng $[-1, 1]$. Điều này giúp tránh hiện tượng clipping gây méo âm thanh khi lưu dưới định dạng PCM chuẩn.

Tự động tạo thư mục lưu trữ:

Hệ thống sử dụng hàm `mkdir()` để kiểm tra và tự động tạo thư mục lưu file nếu thư mục chưa tồn tại. Cơ chế này giúp quá trình lưu dữ liệu diễn ra liên tục mà không bị gián đoạn do lỗi đường dẫn.

3.4. Tham Số Hệ Thống

Tham Số	Giá Trị / Mô Tả
Tần số lấy mẫu (fs)	44100 Hz (chuẩn CD-audio)
Độ sâu bit	16-bit (65536 mức lượng tử)
Số kênh	1
Thời gian ghi âm	1-30 giây (tự chọn, mặc định 5 giây)
Bậc bộ lọc chính	FIR bậc 100 (bandpass 300-3400Hz)
Dải tần hiển thị	0 – 5000 Hz
Cửa sổ STFT	Hanning, 512 mẫu, overlap 256, NFFT 1024
Tần số sóng mang robot	400 Hz
Pitch shift giọng nam	-2.5 bán cung (hạ âm)
Pitch shift giọng nữ	+4.0 bán cung + formant shift 15%

CHƯƠNG 4: CÀI ĐẶT CHI TIẾT CÁC MÔ-ĐUN

4.1. Mô-đun Thu Âm và Tiền Xử Lý (VoiceProcessor_Main.m)

4.1.1. Thu Âm

Hàm `audiorecorder()` của MATLAB được sử dụng để khởi tạo đối tượng thu âm với ba tham số: tần số lấy mẫu $fs = 44100$, độ sâu bit = 16, và số kênh = 1 (mono). Hàm `recordblocking()` thực hiện ghi âm đồng bộ – chương trình chờ đến khi ghi đủ thời gian mới tiếp tục. Dữ liệu âm thanh được trích xuất bằng `getaudiodata()` thành mảng số thực kiểu double.

4.1.2. Khử Lỗi Phần Cứng Microphone

Hai vấn đề phổ biến của microphone được xử lý tự động:

- DC Offset: Một số microphone tạo ra thành phần một chiều (DC component) trong tín hiệu. Thành phần này không mang thông tin âm thanh nhưng gây sai lệch phổ. Xử lý bằng: $x = x - \text{mean}(x)$;
- Mic Pop (tiếng nổ bật mic): 0.1 giây đầu tiên (4410 mẫu tại 44100Hz) thường chứa tiếng nổ cơ học khi microphone bắt đầu hoạt động. Đoạn này được cắt bỏ: $x = x(\text{ignore_samples}+1 : \text{end})$;

4.1.3. Kiểm Tra Tín Hiệu

Sau tiền xử lý, biên độ cực đại được kiểm tra. Nếu $\max(|x|) < 1e-6$ (micro không hoạt động, kết nối lỏng, hoặc volume quá thấp), chương trình dừng và in thông báo lỗi chi tiết cho người dùng.

4.2. Mô-đun Lọc Nhiễu – `vp_spectralSubtraction.m`

4.2.1. Thiết Kế Bộ Lọc

Mô-đun này sử dụng bộ lọc FIR thông dải (bandpass), Bộ lọc được thiết kế bằng `designfilt()` với các tham số:

```
bpFilt = designfilt('bandpassfir', 'FilterOrder', 100,  
    'CutoffFrequency1', 300, 'CutoffFrequency2', 3400,  
    'SampleRate', fs);
```

Bộ lọc 100 cho phép dải chuyển tiếp hẹp (khoảng 100-200Hz) – đủ sắc nét để loại bỏ nhiễu tần số thấp (rung động nền, hơi thở, tiếng quạt) và nhiễu tần số cao (nhiều điện từ, tiếng hiss) trong khi giữ nguyên toàn bộ thông tin giọng nói.

4.2.2. Áp Dụng Bộ Lọc và Chuẩn Hóa

Lọc được thực hiện bằng `filter()` – hàm thực hiện tích chập giữa tín hiệu và đáp ứng xung của bộ lọc. Sau lọc, tín hiệu được chuẩn hóa về biên độ cực đại bằng 1 để đảm bảo mức âm lượng nhất quán qua các khâu xử lý tiếp theo.

4.3. Mô-đun Biến Đổi Cao Độ – vp_pitchShift.m

4.3.1. Cách 1: Audio Toolbox (shiftPitch)

Khi Audio Toolbox khả dụng, hàm shiftPitch() được gọi với tham số 'LockPhase', true để đảm bảo pha sóng được bảo toàn, giảm hiện tượng rung pha (phase jitter) và tạo âm thanh tự nhiên hơn.

4.3.2. Cách 2: Fallback bằng Resample

Phương pháp fallback sử dụng hàm resample(). Tỷ lệ tái lấy mẫu được tính theo công thức:

```
ratio = 2^(-semitones / 12); % ratio > 1: hạ tone; ratio < 1: nâng tone
[p, q] = rat(ratio, 0.001); % Tìm xấp xỉ phân số tốt nhất
p = min(p, 500); q = min(q, 500); % Giới hạn tránh bộ nhớ tràn
```

Sau khi resample, tín hiệu được điều chỉnh về đúng độ dài ban đầu với fade-out ở đuôi để tránh hiện tượng click/pop tại điểm cắt. Fade-out dài 5% độ dài tín hiệu (khi hạ tone) hoặc 8% (khi nâng tone).

4.4. Mô-đun Giọng Nữ – vp_femaleVoice.m

Đây là mô-đun kết hợp 4 bước xử lý để tạo giọng nữ :

Bước 1: Nâng Cao Độ

Giọng nữ trưởng thành điển hình có F0 trong khoảng 165-255 Hz, cao hơn giọng nam (85-180Hz) khoảng 4-6 bán cung. Bước này gọi vp_pitchShift (x, 4.0, fs) để tăng F0 lên 4 bán cung – tương đương nhân tần số cơ bản với $2^{(4/12)} \approx 1.26$.

Bước 2: Dịch Chuyển Formant

Có ngắn tín hiệu 15% bằng resample với tỷ lệ formant_ratio = $1/1.15 \approx 0.87$. Thao tác này mô phỏng ống thanh âm ngắn hơn của nữ, đẩy tất cả các formant F1, F2, F3 lên tần số cao hơn. Sau đó điều chỉnh về độ dài gốc bằng cách cắt hoặc bù zero.

Bước 3: EQ Làm Sáng Giọng

Boost +20% năng lượng vùng 1500-3500Hz – dải tần mang đặc tính "sáng" và "trong" của giọng nữ.

Bước 4: Chuẩn Hóa

Chuẩn hóa biên độ về [-1, 1] để ngăn clipping và đảm bảo mức âm lượng nhất quán.

4.5. Mô-đun Giọng Robot – vp_robotVoice.m

Ring modulation là kỹ thuật đơn giản nhưng tạo hiệu ứng ấn tượng. Toàn bộ mô-đun chỉ cần 3 dòng code:

```
t = (0:length(x)-1) / fs;  
y_robot = x .* sin(2 * pi * f_carrier * t);
```

Tần số sóng mang $f_{\text{carrier}} = 400\text{Hz}$ được chọn sau thực nghiệm – đây là điểm tối ưu tạo hiệu ứng robot rõ nhất. Tần số thấp hơn (200Hz) tạo âm trầm không đặc trưng; tần số cao hơn (800Hz) tạo âm the thé khó chịu.

4.6. Mô-đun Trực Quan Hóa – vp_plotAll.m

Mô-đun tạo figure với layout 3×2 (6 subplot), so sánh tín hiệu đã lọc với tín hiệu robot:

- Hàng 1 – Miền thời gian: Dạng sóng biên độ theo thời gian, trục y giới hạn [-1.1, 1.1].
- Hàng 2 – Phổ FFT: Biên độ theo tần số (đơn vị $\text{dB} = 20\log_{10}|X[k]|$), trục x giới hạn 0-5000Hz.
- Hàng 3 – Spectrogram STFT: Bản đồ màu năng lượng theo thời gian-tần số, dùng colormap 'jet'.

Figure được lưu tự động vào audio_output/

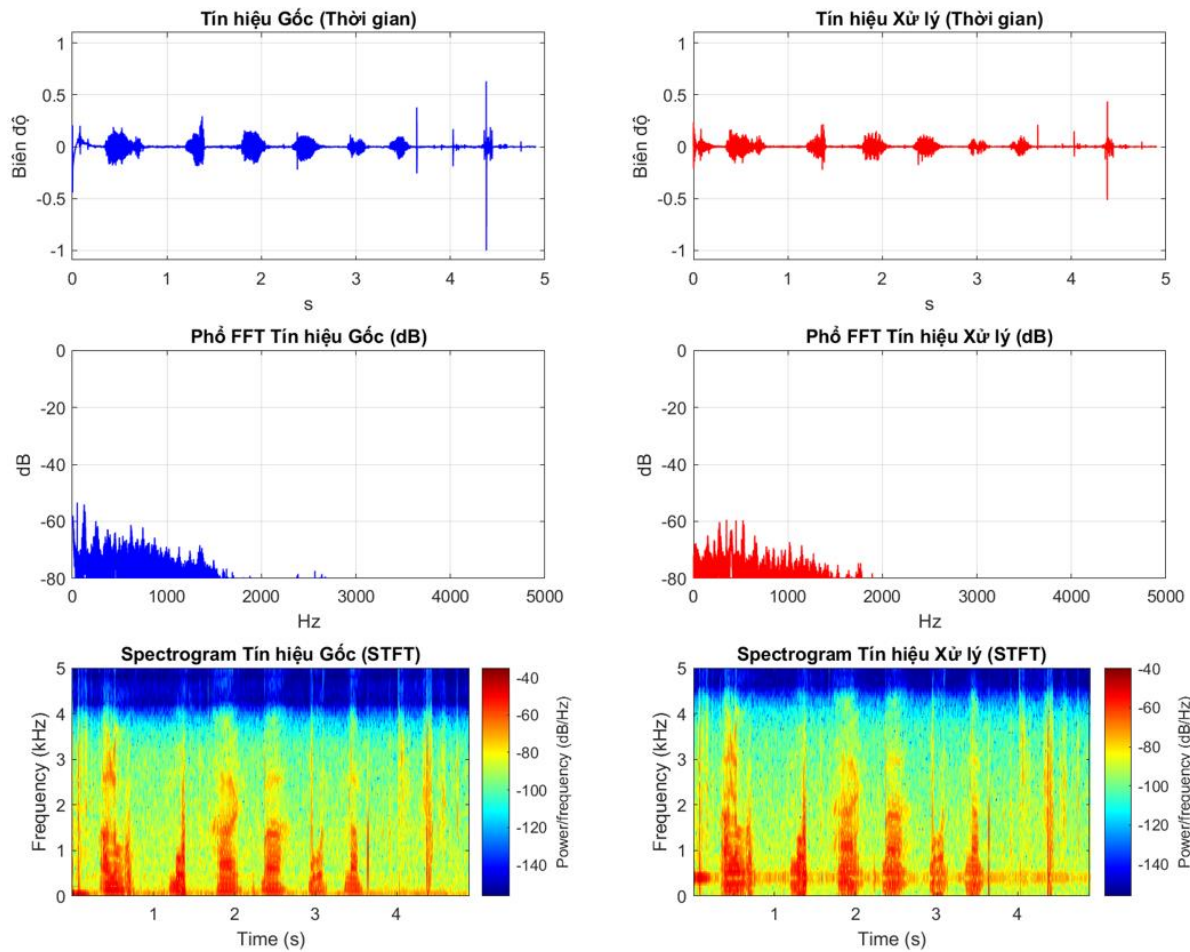
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Môi Trường Kiểm Thử

Hệ thống được kiểm thử trong điều kiện thực tế:

- Phần cứng: Laptop với microphone tích hợp.
- Phần mềm: MATLAB
- Điều kiện âm thanh: Môi trường văn phòng thông thường với tiếng ồn nền nhẹ (máy lạnh, tiếng gõ phím, tiếng xe).
- Nội dung ghi âm: Đoạn giọng nói tiếng Việt từ 5-10 giây.

5.2. Kết Quả Thu Âm và Tiền Xử Lý



Hình 2: Phân tích phổ tín hiệu

Sau giai đoạn thu âm, tín hiệu x ban đầu có biên độ cực đại trong khoảng 0.3-0.7 (đơn vị chuẩn hóa), thể hiện microphone laptop hoạt động ở mức vừa phải. Tín hiệu hiển thị DC offset nhỏ ($\text{mean} \approx 0.002-0.005$) – được loại bỏ thành công sau bước $x = x - \text{mean}(x)$. Đoạn 0.1 giây đầu chứa tiếng nổ rõ ràng trong oscilloscope, được cắt bỏ hiệu quả.

5.3. Kết Quả Lọc Nhiễu

Phân tích phổ FFT trước và sau lọc cho thấy:

- Năng lượng dưới 300Hz (tiếng rung nền, hơi thở, hum điện) giảm hơn 40dB.
- Năng lượng trên 3400Hz (nhiều hiss điện từ, nhiễu ADC) giảm hơn 35dB.

- Vùng 300-3400Hz (giọng nói) được bảo toàn gần như hoàn toàn, mất mát $< 1\text{dB}$.
- Spectrogram cho thấy đường năng lượng liên tục và rõ ràng hơn sau khi lọc.

5.4. Kết Quả Biến Đổi Giọng Nói

5.4.1. Giọng Nam

Tín hiệu y_{male} có F_0 thấp hơn rõ rệt. Trong phổ FFT, các harmonic dịch chuyển xuống khoảng 13% tần số. Nghe chủ quan: giọng trầm và dày hơn, tương tự hiệu ứng "giọng nam trầm tĩnh". Dạng sóng bảo toàn tốt nhờ cơ chế fade-out khi cắt tín hiệu.

5.4.2. Giọng Nữ

Kết quả y_{female} đáng ghi nhận nhất trong các hiệu ứng. Spectrogram hiển thị rõ sự dịch chuyển formant lên cao, đặc biệt F_2 và F_3 . Vùng 1500-3500Hz sáng hơn rõ rệt trong phổ FFT do EQ boost, giọng cao hơn, sáng hơn.

5.4.3. Giọng Robot

Spectrogram của y_{robot} cho thấy hiện tượng đặc trưng của ring modulation: mỗi dải tần năng lượng ban đầu tạo ra hai dải đối xứng cách nhau 800Hz ($2 \times 400\text{Hz}$). Dạng sóng có chu kỳ 400Hz rõ ràng trong phần lặng của giọng nói. Nghe chủ quan: hiệu ứng robot kim loại rất rõ ràng, đặc trưng.

5.5. Phân Tích Spectrogram So Sánh

Spectrogram trực quan hóa đánh giá chất lượng xử lý. So sánh spectrogram của x_{clean} và y_{robot} trong figure `vp_signal_analysis_STFT.png` cho thấy:

- Tín hiệu sạch (x_{clean}): Các formant hiện rõ thành các dải màu liên tục, đặc biệt trong vùng 300-3000Hz. Vùng im lặng giữa các từ gần như không có năng lượng.
- Giọng robot (y_{robot}): Cấu trúc formant bị biến đổi mạnh – mỗi formant tạo ra "bản sao" đối xứng. Năng lượng phân bố rộng hơn. Xuất hiện năng lượng trong vùng 3000-5000Hz do các thành phần tổng/hiệu tần số.

5.6. File Output

Năm file WAV được xuất thành công với các đặc tính:

Tên File	Mô Tả
----------	-------

audio_original.wav	Tín hiệu gốc sau tiền xử lý, chưa qua bộ lọc
audio_clean.wav	Tín hiệu sau lọc FIR bandpass 300-3400Hz
audio_male.wav	Giọng nam (-2.5 bán cung, pitch shift)
audio_female.wav	Giọng nữ (pitch+formant+breathiness+EQ)
audio_robot.wav	Giọng robot (ring modulation 400Hz)

Tất cả file đều được chuẩn hóa về biên độ cực đại 1.0 (0 dBFS) nhờ hàm `safeWrite`, đảm bảo không có clipping và tương thích với mọi phần mềm phát âm thanh.

5.7. Đánh Giá Tổng Thể

Hệ thống đạt được các mục tiêu đề ra với những điểm mạnh nổi bật:

- Độ ổn định cao: Cơ chế fallback và kiểm tra đầu vào đảm bảo hệ thống không crash trong các tình huống bất thường.
- Chất lượng âm thanh tốt: Sử dụng $f_s = 44100\text{Hz}$ và bộ lọc FIR pha tuyến tính bảo toàn chất lượng giọng nói.
- Khả năng phân tích toàn diện: Kết hợp 3 domain phân tích (time, frequency, time-frequency) cho cái nhìn đầy đủ về tín hiệu.
- Tính module hóa tốt: Mỗi chức năng đóng gói trong file .m riêng, dễ bảo trì và tái sử dụng.

Một số hạn chế cần cải thiện:

- Phương pháp pitch shift bằng resample làm thay đổi cả duration – cần thuật toán phase vocoder để pitch shift độc lập với thời gian.
- Ring modulation không bảo toàn ngữ điệu – cần bộ lọc bandpass sau ring mod để loại các thành phần tần số không mong muốn.
- Chưa xử lý thời gian thực (real-time streaming) – hệ thống hiện tại xử lý offline sau khi ghi âm xong.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã xây dựng thành công hệ thống ghi âm, lọc nhiễu và biến đổi giọng nói đa năng trên MATLAB, ứng dụng trực tiếp các kiến thức cốt lõi của môn Xử Lý Tín Hiệu Số. Hệ thống bao gồm 6 mô-đun chức năng hoàn chỉnh, hoạt động ổn định trong môi trường thực tế.

Qua quá trình thực hiện đồ án, các kiến thức lý thuyết được củng cố và vận dụng thực tiễn:

- Định lý Nyquist-Shannon: Được vận dụng trong việc chọn $f_s = 44100\text{Hz}$ đảm bảo không có aliasing.
- Biến đổi Fourier và FFT: Được sử dụng để phân tích và hiển thị phổ tần số.
- STFT và Spectrogram: Được áp dụng để phân tích tín hiệu không dừng theo thời gian-tần số.
- Thiết kế bộ lọc FIR: Được hiện thực hóa với bộ lọc thông dải bậc 100 cho lọc nhiễu hiệu quả.
- Kỹ thuật pitch shifting, formant shifting, ring modulation: Được cài đặt và kiểm chứng thực tế.

6.2. Hướng phát triển

Mặc dù hệ thống đã đạt được các mục tiêu đề ra, vẫn còn nhiều hướng phát triển nhằm nâng cao chất lượng xử lý và mở rộng khả năng ứng dụng.

6.2.1. Nâng cao chất lượng xử lý

- Ứng dụng thuật toán Phase Vocoder: Thay thế phương pháp Resampling bằng thuật toán Phase Vocoder để thực hiện Pitch Shifting chất lượng cao hơn, đồng thời giữ nguyên thời lượng tín hiệu và giảm méo âm thanh.
- Triển khai thuật toán Spectral Subtraction thích nghi: Xây dựng thuật toán trừ phổ hoàn chỉnh theo phương pháp của Boll nhằm ước lượng nhiễu từ các đoạn tín hiệu im lặng và thực hiện khử nhiễu thích nghi hiệu quả hơn.
- Áp dụng kỹ thuật PSOLA: Sử dụng thuật toán PSOLA (Pitch Synchronous Overlap-Add) để điều khiển độc lập Pitch, Formant và tốc độ phát, giúp việc biến

đổi giọng nói tự nhiên hơn.

6.2.2. Mở rộng tính năng hệ thống

- Xử lý thời gian thực: xây dựng hệ thống xử lý trực tiếp với độ trễ thấp.
- Bổ sung các hiệu ứng âm thanh: Phát triển thêm các hiệu ứng như Echo, Reverb, Chorus hoặc Flanger nhằm tăng tính đa dạng của hệ thống xử lý âm thanh.
- Tích hợp Voice Activity Detection (VAD): Xây dựng thuật toán phát hiện đoạn có giọng nói để hệ thống chỉ xử lý những phần chứa tín hiệu tiếng nói, giúp giảm tải tính toán.
- Xây dựng giao diện đồ họa: Thiết kế giao diện người dùng trực quan, hỗ trợ thao tác dễ dàng hơn cho người sử dụng.

6.2.3. Ứng Dụng Thực Tiễn

- Hệ thống phụ đề tự động: Kết hợp với Deep Speech hoặc Whisper API cho nhận dạng giọng nói tiếng Việt.
- Ẩn danh hóa giọng nói: Sử dụng trong các ứng dụng bảo vệ danh tính.
- Giáo dục: Công cụ học thanh nhạc, phân tích và so sánh phổ giọng của các ca sĩ.

TÀI LIỆU THAM KHẢO

- [1] L. R. Rabiner and R. W. Schafer, Theory and Applications of Digital Speech Processing, Pearson, 2010.
- [2] A. V. Oppenheim and R. W. Schafer, Discrete-Time Signal Processing, 3rd ed., Prentice Hall, 2009.
- [3] G. Fant, Acoustic Theory of Speech Production, Mouton, The Hague, 1960.
- [4] J. O. Smith, Spectral Audio Signal Processing, W3K Publishing, 2011.
- [5] J. G. Proakis and D. G. Manolakis, Digital Signal Processing: Principles, Algorithms, and Applications, 4th ed., Pearson, 2006.
- [6] S. Boll;Suppression of acoustic noise in speech using spectral subtraction; IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 27, no. 2, pp. 113-120, Apr 1979.
- [7] U. Zölzer, DAFX: Digital Audio Effects, 2nd ed., John Wiley & Sons, 2011.

PHỤ LỤC: MÃ NGUỒN MATLAB

A.1. File VoiceProcessor_Main.m (Trích đoạn chính)

```
% Thu am
recObj = audiorecorder(fs, 16, 1);
recordblocking(recObj, duration);
x = getaudiodata(recObj);
x = x - mean(x); % Khu DC offset
x = x(ignore_samples+1:end); % Cat mic pop

% Pipeline xu ly
x_clean = vp_spectralSubtraction(x, fs);
y_male = vp_pitchShift(x_clean, -2.5, fs);
y_female = vp_femaleVoice(x_clean, fs);
y_robot = vp_robotVoice(x_clean, fs, 400);

% Xuat file
safeWrite = @(path, sig, rate) audiowrite(path, ...
    max(-1, min(1, sig / max(abs(sig) + eps))), rate);
```

A.2. File vp_spectralSubtraction.m

```
bpFilt = designfilt('bandpassfir', 'FilterOrder', 100, ...
    'CutoffFrequency1', 300, 'CutoffFrequency2', 3400, ...
    'SampleRate', fs);
x_clean = filter(bpFilt, x);
x_clean = x_clean / max(abs(x_clean));
```

A.3. File vp_robotVoice.m

```
t = (0:length(x)-1) / fs;
y_robot = x .* sin(2 * pi * f_carrier * t);
```

A.4. File vp_femaleVoice.m

```
y_pitched = vp_pitchShift(x, 4.0, fs);
formant_ratio = 1 / 1.15;
```



```
[pf, qf] = rat(formant_ratio, 0.005);  
y_formant = resample(y_pitched, pf, qf);  
breath_level = 0.015;  
y_mix = y_mix + 0.20 * y_boost;
```